



Rapport de stage

Développement web Angular sur une application interne de gestion de flotte

BUT Informatique - 2e année

Parcours Réalisation d'applications : conception, développement, validation

Année universitaire 2025-2026

Étudiant

Nathanaël Bayard

Entreprise d'accueil

Amiltone

Adresse de l'entreprise

76 boulevard du 11 Novembre 1918, 69100 Villeurbanne

Période de stage

Du 28 avril 2026 au 26 juin 2026

Maître de stage

Freddy Louvier, responsable Nexus

Tuteur enseignant

Anthony BUSSON

IUT

IUT Lyon 1, Université Claude Bernard Lyon 1, 76 boulevard Niels Bohr, 69100 Villeurbanne

Fiche technique

Rubrique	Informations
Nom de l'entreprise	Amiltone
Activité de l'entreprise	ESN spécialisée dans la conception, le développement, les tests et le déploiement de solutions numériques.
Intitulé du sujet ou des sujets	Développement web Angular sur Flotto, application de gestion de flotte de véhicules.
Sujet(s) indépendant(s) ou s'inscrit dans un projet plus large ?	Sujet inscrit dans Flotto, projet développé au sein du Nexus d'Amiltone.
Le travail répond à quels besoins ?	Amélioration, maintenance et qualité du projet Flotto.
Il doit atteindre quels objectifs ?	Monter en compétences, comprendre l'existant et traiter des tickets Angular.
A qui le travail est-il destiné ?	À l'équipe Flotto et aux utilisateurs de l'application, notamment des clients externes.
Quel sera son devenir ?	Les contributions validées sont intégrées au projet Flotto.
Quelles sont les contraintes de temps ?	Stage de dix semaines, organisé par sprints, avec montée en compétences progressive.
Y a-t-il des informaticiens dans l'entreprise ?	Oui, notamment au sein du Nexus et de l'équipe Flotto.
Est-ce que vous travaillez seul ?	Non. Je travaille avec l'équipe Flotto, tout en gardant une part d'autonomie sur mes tickets.
Avec un autre stagiaire, un alternant ?	Pas directement sur Flotto, même si d'autres stagiaires ou alternants sont présents chez Amiltone.
Sur le même projet ou sur des projets différents ?	Je travaille principalement sur Flotto ; d'autres membres du Nexus interviennent sur d'autres projets.

Rubrique	Informations
Avec une équipe de l'entreprise ? Quel est le matériel utilisé ?	Oui, avec l'équipe Flotto au sein du Nexus. PC portable Amilstone sous Windows, casque, VS Code, WSL, Docker, GitLab, Jira et Teams.
Quels sont les langages ?	TypeScript, HTML et SCSS pour la partie web Angular.
Avez-vous dû apprendre un nouveau langage ?	Pas un nouveau langage au sens strict : j'ai surtout dû apprendre Angular, un framework, et approfondir TypeScript.
Quelle méthode ?	Méthode agile : tickets Jira, sprints, daily, démonstrations, merge requests et revues de code.
A la fin du stage, quelle partie du travail doit être terminée ?	Les tickets attribués doivent être terminés et validés selon leur nature.
Avez-vous eu des contraintes ou des facilités pour la rédaction de votre rapport de stage ?	Rédaction surtout réalisée à la maison en parallèle du stage.

Remerciements

Je tiens tout d'abord à remercier Amiltone pour son accueil au sein de l'agence de Villeurbanne et pour la confiance accordée pendant ce stage. Je remercie plus particulièrement Freddy Louvier, mon maître de stage, pour m'avoir permis d'intégrer l'équipe Nexus et de découvrir le projet Flotto dans un contexte professionnel réel.

Je remercie également les membres de l'équipe, avec lesquels j'ai pu échanger au quotidien, et notamment Swan DAPHNE, qui travaillait également dans les bureaux de Lyon. Leur disponibilité, leurs explications et les démonstrations produites m'ont aidé à comprendre progressivement l'organisation du projet, les outils utilisés et les attentes d'une équipe de développement en entreprise.

Enfin, je remercie l'IUT Lyon 1 et mon tuteur enseignant, Anthony BUSSON, pour le suivi du stage et pour la formation qui m'a permis d'aborder cette première expérience professionnelle avec des bases techniques et méthodologiques utiles.

Sommaire

Introduction	7
1 Contexte professionnel et environnement de stage	8
1.1 Présentation générale d'Amiltone	8
1.2 Service, équipe et place du stagiaire	9
1.3 Environnement technique et méthodes de travail	10
1.4 Intégration et montée en compétences initiale	11
2 Missions confiées et démarche de réalisation	13
2.1 Sujet général du stage	13
2.2 Mission 1 - Refactorisation des champs de date et d'heure en composants réutilisables	13
2.2.1 Contexte et objectif	13
2.2.2 Démarche adoptée	15
2.2.3 Difficultés, solutions et état d'avancement	16
2.2.4 Retour de production et hotfix	16
2.3 Mission 2 - Création et uniformisation des tableaux de gestion	17
2.3.1 Enjeux communs et démarche générale	17
2.3.2 Création du tableau des sites de rattachement	18
2.3.3 Évolution de la page Membres et uniformisation visuelle	20
2.3.4 Résultats et apprentissages	22
2.4 Missions ponctuelles ou récurrentes	22
3 Organisation du travail, résultats et apports	23
3.1 Gestion du projet et calendrier des missions	23
3.2 Résultats obtenus	24
3.3 Apports pour l'entreprise	25
3.4 Limites et pistes de suite	25
4 Bilan de compétences et projet professionnel	26
4.1 Compétences techniques mobilisées	26
4.2 Compétences transversales développées	26
4.3 Difficultés formatrices et progression personnelle	27
4.4 Lien avec la formation et le projet professionnel	28
Conclusion	29
Sources	30

Glossaire

Terme	Définition
Nexus	Pôle interne d'Amiltone dans lequel se déroule le stage.
Flotto	Application de gestion de flotte de véhicules sur laquelle porte le stage.
Angular	Framework web utilisé pour développer des applications front-end structurées.
TypeScript	Langage basé sur JavaScript avec un système de typage statique, utilisé notamment avec Angular.
Jira	Outil de suivi de tickets et d'organisation du travail en équipe.
GitLab	Plateforme de gestion de code source, de branches, de merge requests et de pipelines CI/CD.
Confluence	Outil de documentation utilisé pour centraliser les informations projet et les procédures internes.
Merge request	Demande d'intégration d'une branche de code dans une autre, accompagnée d'une revue de code.
CI/CD	Ensemble de pratiques permettant d'automatiser l'intégration, les tests et le déploiement d'un projet.
DevOps	Pratiques et rôles liés à l'automatisation, au déploiement et à la fiabilité des environnements applicatifs.
Hotfix	Correction urgente destinée à résoudre rapidement un bug important, notamment lorsqu'il est présent en production.

Terme	Définition
Sprint	Période de travail courte pendant laquelle une équipe réalise un ensemble de tickets priorités.
Daily	Point quotidien court permettant de partager l'avancement, les blocages et les besoins d'aide.
Definition of Done	Ensemble de critères permettant de considérer qu'un ticket est réellement terminé.
Staging	Environnement de validation stable utilisé avant une mise en préproduction ou en production.
Préproduction	Environnement proche de la production utilisé pour les dernières validations.
QA	Assurance qualité, ensemble des activités visant à vérifier qu'un produit respecte les attentes.
RGPD	Règlement général sur la protection des données.
WSL	Windows Subsystem for Linux, environnement permettant d'utiliser un système Linux sous Windows.
MinIO	Solution de stockage objet compatible avec le protocole S3.
TypeORM	Outil facilitant la communication entre une application TypeScript et une base de données relationnelle.
Playwright	Outil permettant d'automatiser des tests sur une application web.

Introduction

Dans le cadre de ma deuxième année de BUT Informatique à l'IUT Lyon 1, parcours Réalisation d'applications, j'effectue un stage au sein d'Amiltone, une entreprise de services du numérique située à Villeurbanne. Ce stage se déroule du 28 avril 2026 au 26 juin 2026, pour une durée totale de dix semaines.

Amiltone accompagne des entreprises dans la conception et le développement de solutions numériques. Ses activités couvrent notamment le développement web et mobile, la data, l'intelligence artificielle, l'UX/UI, les tests, le DevOps et la cybersécurité. Pour ce stage, j'ai été intégré au Nexus, un pôle interne consacré à la formation, à l'innovation et au développement de produits logiciels. Mon travail s'inscrit plus précisément dans le projet Flotto, une application de gestion de flotte de véhicules.

Au départ, j'imaginais plutôt découvrir le développement mobile avec Flutter, car c'était l'orientation prévue dans la convention. Le contexte du projet a cependant changé : la personne chargée de cette partie ayant quitté l'équipe, mon encadrement a préféré m'intégrer sur la partie web avec Angular. Ce changement m'a d'abord demandé un temps d'adaptation, car Angular était une technologie que je n'avais pas encore étudiée en cours. Avec le recul, il s'est finalement révélé cohérent avec mon parcours RA et avec les compétences attendues en développement d'applications.

Avant ce stage, j'avais surtout une vision scolaire du développement et de l'organisation d'un projet. Je voulais donc voir plus concrètement comment fonctionne une ESN, comment une équipe suit ses tickets et comment des outils comme Jira, GitLab ou Docker sont utilisés au quotidien. Je souhaitais aussi vérifier ma capacité à m'adapter à un projet déjà avancé, avec un existant important et des standards de qualité à respecter.

Ce rapport présente d'abord le contexte professionnel du stage, l'entreprise d'accueil, l'équipe et l'environnement technique. Il décrit ensuite l'organisation du travail, les premières missions confiées et la démarche de montée en compétences. Enfin, il proposera un bilan des compétences développées pendant le stage et un retour sur l'apport de cette expérience pour la suite de mon parcours.

Chapitre 1

Contexte professionnel et environnement de stage

1.1 Présentation générale d'Amiltone

Amiltone est une ESN, c'est-à-dire une entreprise de services du numérique. Son activité consiste à accompagner des organisations dans leurs projets informatiques, depuis l'analyse du besoin jusqu'à la mise en place de solutions adaptées. L'entreprise intervient sur plusieurs domaines du numérique : développement web et mobile, data, intelligence artificielle, UX/UI, gestion de projet, tests logiciels, DevOps et cybersécurité.

Cette diversité d'activités montre que l'entreprise ne se limite pas à la production de code. Elle accompagne aussi les choix techniques, la qualité, l'expérience utilisateur, la validation et le déploiement des solutions. Dans le cadre d'un stage de BUT Informatique, cet environnement est intéressant car il permet d'observer plusieurs dimensions d'un projet logiciel : la technique, l'organisation, la communication entre les membres de l'équipe et la prise en compte des besoins métier.

Au sein de cette organisation, le Nexus occupe une place particulière. Il ne correspond pas seulement à une équipe de développement classique : il sert aussi d'espace de montée en compétences, de capitalisation et d'expérimentation. Les collaborateurs peuvent y travailler sur des produits internes, renforcer leurs compétences techniques ou préparer une future mission client. Cette situation rend la documentation, la transmission et la qualité du code importantes, car les membres peuvent changer de projet ou rejoindre une mission externe.

Le stage se déroule dans l'agence de Villeurbanne, située au 76 boulevard du 11 Novembre 1918. Amiltone compte plus de 400 collaborateurs au total. Dans ce rapport, je ne retiendrais que ce chiffre global plutôt qu'un effectif détaillé par agence, afin de m'appuyer uniquement sur des informations suffisamment fiables et utiles à la compréhension du stage.

Flotto est déployée en production et utilisée par des clients externes. Ceux-ci ne seront pas cités nominativement dans le rapport ; les utilisateurs et les données présentés dans les exemples et les captures d'écran seront fictifs.

1.2 Service, équipe et place du stagiaire

J'ai été intégré au Nexus, un pôle interne d'Amiltone qui regroupe plusieurs produits logiciels. Le Nexus remplit plusieurs fonctions : il permet aux collaborateurs de monter en compétences, d'expérimenter de nouvelles approches, de documenter les connaissances et de contribuer à des applications utiles à l'entreprise ou potentiellement valorisables à l'extérieur. Il accueille donc des profils variés, parfois présents de façon temporaire avant un départ en mission client.

Le Nexus ne porte pas un seul projet. Plusieurs applications y sont référencées, par exemple Flotto, TicknTrip, Highlight ou Up!. Tous ces produits n'ont pas le même niveau d'activité, mais ils montrent que le Nexus fonctionne comme un espace produit et technique commun. Cette organisation explique la présence de pratiques partagées : documentation Confluence, suivi des tickets, revues de code, tests, environnements de validation et rituels agiles.

Le projet sur lequel je travaille principalement s'appelle Flotto. Il s'agit d'une application de gestion de flotte de véhicules. Elle centralise plusieurs besoins : suivi des véhicules, trajets, incidents, documents, utilisateurs, réservations et opérations de maintenance. Le projet possède déjà un existant important, ce qui rend la phase de prise en main particulièrement importante.

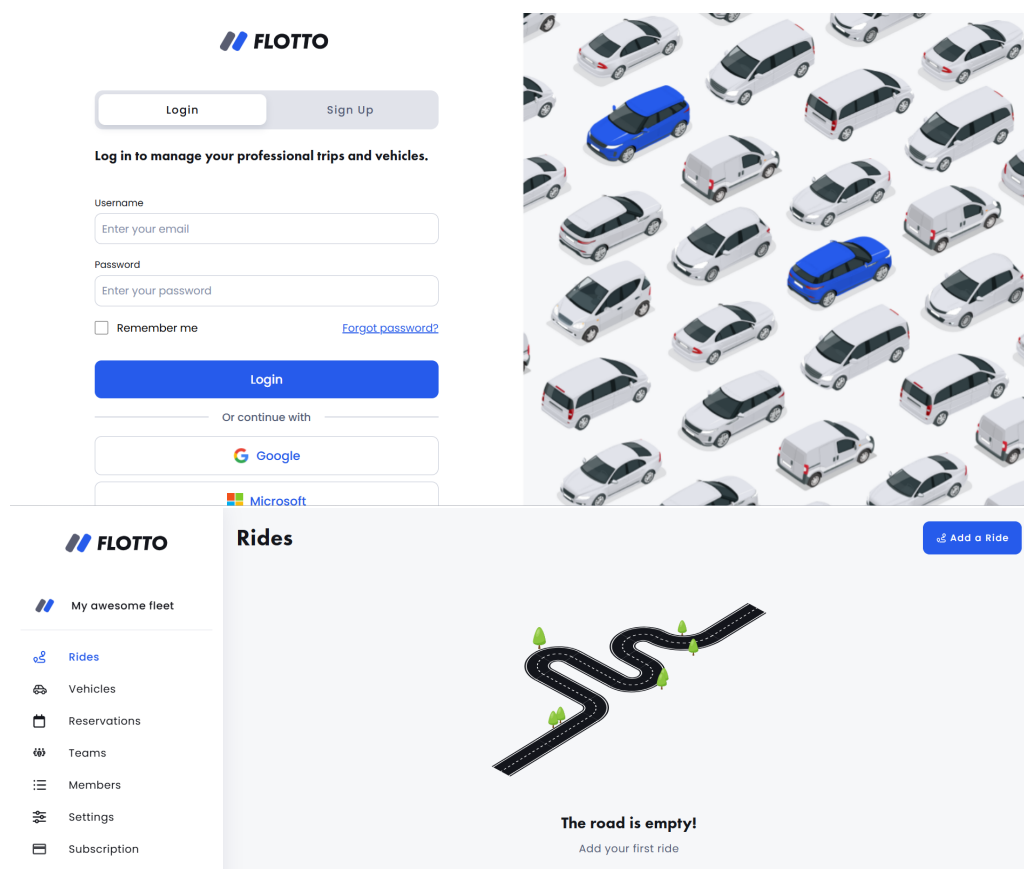


Fig. 1.1 – Exemples d'écrans de Flotto : écran de connexion et vue vide du module trajets.

L'équipe projet est composée d'environ huit personnes, même si cet effectif peut varier selon les missions chez les clients et les disponibilités de chacun. Les profils autour de moi sont principalement des développeurs avec des spécialités différentes : développement web, développement back-end, Node.js, Angular, Java, React ou encore .NET. Selon les périodes, certains rôles comme la QA, le pilotage fonctionnel ou le développement mobile peuvent être moins présents, ce qui oblige l'équipe à porter une attention particulière à la qualité, aux tests et à la documentation.

Ma place dans l'équipe est celle d'un stagiaire en montée en compétences. Je ne suis pas arrivé sur un projet commencé de zéro, mais sur une application déjà structurée, avec ses habitudes, son architecture, ses outils et ses règles de qualité. Mon rôle consiste donc d'abord à comprendre l'existant, puis à traiter progressivement des tickets adaptés à mon niveau, notamment autour de la refactorisation et de l'amélioration de la qualité du code.

Le travail se fait sur site, à l'agence de Villeurbanne. Mes horaires habituels sont de 9 h à 17 h 30. L'équipe suit une organisation inspirée des méthodes agiles. Un point quotidien, ou daily, a lieu chaque matin à 9 h 30. Il dure environ quinze minutes et permet à chacun d'indiquer ce qu'il a fait la veille, les difficultés rencontrées et les besoins d'aide. Une réunion hebdomadaire a également lieu le mercredi : un membre de chaque équipe y présente l'avancement de la semaine précédente. En fin de sprint, l'équipe réalise une démonstration de l'application avec les tickets terminés, puis une réunion de chiffrage permet d'estimer les tickets à venir en points d'effort.

1.3 Environnement technique et méthodes de travail

Sur le projet Flotto, la partie sur laquelle je suis progressivement intégré concerne principalement le développement web avec Angular. Les langages utilisés sont donc TypeScript, HTML et SCSS. Angular étant un framework structuré, sa prise en main demande de comprendre à la fois l'organisation des composants, les services, les templates, le typage TypeScript et les conventions du projet. Les bonnes pratiques internes encouragent notamment une organisation par modules métier, des composants lisibles, une séparation claire entre la logique d'affichage et la logique métier, ainsi qu'une attention particulière aux tests et à la maintenabilité du SCSS.

L'environnement technique ne se limite pas au front-end. Le projet s'appuie aussi sur des outils et technologies utilisés par l'équipe, notamment GitLab pour la gestion du code, des branches, des merge requests et des pipelines CI/CD, Jira pour le suivi des tickets, Docker et WSL pour l'environnement de développement, Node.js pour l'écosystème JavaScript, ainsi que PostgreSQL et MinIO pour certaines données et fichiers. D'autres outils existent dans l'environnement de l'agence, comme Squash pour les tests fonctionnels, Playwright pour l'automatisation de certains tests et SonarQube. Je ne les utilise pas tous directement à ce stade, mais ils font partie de l'écosystème technique que je découvre.

La gestion du travail se fait par tickets. Un besoin, une correction ou une tâche technique est formalisé dans Jira, puis intégré au backlog ou à un sprint selon les priorités. Le ticket passe ensuite par plusieurs états : préparation, développement, merge request, intégration sur les environnements de validation, puis livraison. Cette organisation m'aide à comprendre la différence entre une tâche scolaire, souvent isolée, et une tâche professionnelle, qui s'insère dans un backlog, un sprint, une équipe et une application existante.

Le workflow de développement s'appuie également sur GitLab. Lorsqu'un ticket est traité, le développeur crée une branche, réalise les modifications, effectue des vérifications locales, puis ouvre une merge request. Cette merge request permet à un autre membre de l'équipe de relire le code, de discuter les choix effectués et de demander des corrections si nécessaire. Le travail n'est donc pas considéré comme terminé uniquement lorsque le code compile : il doit être lisible, testé, cohérent avec l'existant et placé dans le bon état du workflow.

La validation occupe aussi une place importante. Avant de proposer une modification, l'équipe accorde de l'importance aux tests et à la qualité du code. Des tests manuels, des vérifications sur l'application, des cas de test dans Squash ou des tests automatisés avec Playwright peuvent être mobilisés selon le type de modification. Les environnements de développement, de staging, de préproduction et de production structurent aussi les livraisons et limitent les risques de régression.

1.4 Intégration et montée en compétences initiale

Les premiers jours du stage ont été consacrés à la découverte de l'entreprise et à l'installation de mon environnement de travail. L'arrivée au Nexus est structurée par des documents d'accueil, des accès aux outils et une documentation Confluence importante. J'ai commencé par lire ces ressources afin de comprendre le fonctionnement général d'Amiltone, les produits du Nexus, les rituels d'équipe et les règles de travail partagées.

Cette phase de documentation était nécessaire, car le Nexus repose beaucoup sur la transmission. Les collaborateurs peuvent changer de projet ou partir en mission client ; les connaissances doivent donc être conservées dans des supports accessibles. Pour un stagiaire, cette culture documentaire aide à comprendre l'organisation, mais demande aussi de savoir trier les informations : certaines pages sont anciennes, certaines concernent d'autres produits, et toutes ne sont pas directement utiles pour mes missions.

Dans un second temps, j'ai intégré l'équipe Flotto. La prise en main du projet a demandé un temps d'adaptation, car l'application comporte déjà beaucoup d'éléments : structure Angular, composants existants, organisation du code, tickets Jira, environnement de développement, workflows GitLab et habitudes d'équipe. L'apprentissage d'Angular a donc été mené en parallèle de la découverte du produit et de ses règles métier.

Cette phase d'intégration m'a fait comprendre l'importance de l'autonomie dans un environnement professionnel. Mon maître de stage étant assez occupé, je dois apprendre à chercher par moi-même, à lire la documentation, à analyser le code existant et à formuler des questions précises lorsque je suis bloqué. Je peux cependant demander de l'aide aux membres de l'équipe, en particulier via Teams ou directement dans les bureaux.

Les premières semaines ont ainsi été organisées autour de trois objectifs : comprendre le fonctionnement général d'Amiltone et du Nexus, prendre en main le projet Flotto et commencer à contribuer par de petits tickets. Cette progression me permet d'avancer sans masquer les difficultés liées à la découverte simultanée d'une nouvelle technologie, d'un projet complexe et d'un environnement professionnel.

Chapitre 2

Missions confiées et démarche de réalisation

2.1 Sujet général du stage

Le sujet du stage porte sur ma participation au développement web de Flotto, une application de gestion de flotte de véhicules, avec une montée en compétences progressive sur Angular.

Ce sujet n'est pas exactement celui envisagé au départ. Le passage de Flutter à Angular s'explique par le contexte de l'équipe : il n'était pas pertinent de me laisser seul sur un périmètre mobile alors que j'étais encore en phase de découverte. Le stage a donc été recentré sur le développement web afin de permettre une progression plus encadrée et plus cohérente avec les besoins immédiats du projet.

Les missions confiées s'inscrivent dans une application existante et partagent plusieurs contraintes : respecter l'architecture et les conventions du projet, préserver les comportements déjà en place et vérifier les risques de régression avant l'intégration des modifications.

Les objectifs principaux sont la montée en compétences, l'apprentissage de nouvelles technologies et l'intégration dans un vrai projet logiciel. Pour l'entreprise, l'enjeu est aussi de me permettre de contribuer progressivement aux sprints, tout en respectant les exigences de qualité du projet. Deux missions significatives sont présentées dans ce rapport : la refactorisation de champs de formulaire réutilisables, puis la création et l'uniformisation de plusieurs tableaux de gestion.

2.2 Mission 1 – Refactorisation des champs de date et d'heure en composants réutilisables

2.2.1 Contexte et objectif

L'une des premières missions suffisamment représentatives pour le rapport a concerné les champs de date et d'heure de l'application Flotto. Elle correspond au ticket Jira *FLT-1517*, intitulé *[FRONT] Refactoriser les inputs date/heure en composants réutilisables*, traité pendant le sprint 6. Ces champs apparaissent dans plusieurs parties du produit, notamment dans les formulaires liés aux véhicules, aux trajets, aux réservations, à la maintenance et aux paramètres. Ils sont donc utilisés dans des parcours différents, avec des contraintes de saisie, de validation et d'affichage qui ne sont pas toujours exactement les mêmes.

Avant cette intervention, ces champs reposaient directement sur les composants Angular Material `mat-datepicker` et `mat-timepicker` dans neuf templates de l'application. Cette approche fonctionnait, mais elle entraînait une duplication importante de code. Les mêmes logiques étaient répétées à différents endroits, avec des classes CSS, des conventions de nommage et une gestion des erreurs parfois différentes selon les formulaires. À terme, cette dispersion rendait les évolutions plus coûteuses et augmentait le risque d'incohérences visuelles ou comportementales.

Fig. 2.1 – Exemples de champs date et heure utilisés dans deux formulaires de Flotto. Les encadrés rouges indiquent les éléments concernés par le ticket de refactorisation.

L'objectif était donc de créer deux composants partagés dans le module commun de l'application : `app-date-input` pour les champs de date et `app-time-input` pour les champs d'heure. Ces composants devaient pouvoir être utilisés comme des champs Angular classiques dans les formulaires réactifs, notamment avec `formControlName`. Ils devaient également prendre en charge des paramètres utiles comme `min`, `max` ou `disabled`, tout en harmonisant le rendu visuel, les états de focus et les états d'erreur.

Cette mission ne consistait pas à ajouter une nouvelle fonctionnalité visible pour l'utilisateur final. Il s'agissait plutôt d'une refactorisation technique : améliorer la maintenabilité de l'existant sans modifier le comportement métier attendu. Ce type de travail est important dans une application déjà avancée comme Flotto, car il prépare les futures évolutions et réduit la dette technique.

2.2.2 Démarche adoptée

J'ai commencé par identifier les endroits où les champs de date et d'heure étaient utilisés dans l'application. Cette étape d'analyse était nécessaire pour éviter de créer un composant trop limité à un seul formulaire. Elle m'a aussi obligé à parcourir plusieurs dossiers et à mieux comprendre l'organisation du projet Angular : modules métier, composants de formulaire, templates HTML, fichiers SCSS et liaisons avec les formulaires réactifs.

Le choix technique principal a été d'utiliser l'interface `ControlValueAccessor`. Dans Angular, cette interface permet à un composant personnalisé de se comporter comme un champ de formulaire standard. Grâce à cette approche, les nouveaux composants peuvent être utilisés avec `formControlName`, sans obliger les formulaires existants à changer leur logique métier. Le composant partagé devient responsable de la présentation et de la gestion de l'entrée utilisateur, tandis que le formulaire conserve sa logique de validation et ses règles propres.

J'ai également choisi de conserver les composants Angular Material existants à l'intérieur des nouveaux composants. Cette décision limitait les risques, car il ne s'agissait pas de remplacer complètement le fonctionnement technique des sélecteurs de date et d'heure, mais de les encapsuler dans une couche plus homogène. L'objectif était donc de centraliser le comportement commun tout en restant cohérent avec le reste du projet.

La mise en place s'est faite progressivement. Après la création des deux composants partagés, j'ai remplacé les anciennes implémentations dans les formulaires concernés, puis adapté les imports, les bindings Angular et les règles SCSS nécessaires. À chaque remplacement, il fallait vérifier que le formulaire continuait à recevoir la bonne valeur, que les contraintes de dates étaient bien appliquées et que les états d'erreur restaient cohérents.

Les vérifications ont été menées en local, puis sur l'environnement de développement après intégration. Elles ont notamment porté sur les points suivants :

Élément vérifié	Objectif de la vérification
Sélection via le calendrier ou le sélecteur d'heure	Contrôler que les composants Angular Material restaient utilisables après encapsulation.
Saisie manuelle au clavier	Vérifier que l'utilisateur pouvait toujours saisir une valeur sans passer uniquement par le calendrier.
Contraintes <code>min</code> et <code>max</code>	S'assurer que les limites de dates continuaient à être respectées.
État <code>disabled</code>	Vérifier que les champs désactivés ne pouvaient pas être modifiés.

Élément vérifié	Objectif de la vérification
États de focus et d'erreur	Contrôler l'affichage visuel des bordures, messages et retours utilisateur.

Cette phase de test a concerné plusieurs formulaires, notamment la création de trajet, les réservations, la maintenance, la fiche véhicule et les paramètres. Elle a montré qu'une refactorisation apparemment simple doit être testée dans plusieurs contextes, car un composant partagé peut avoir des effets différents selon la manière dont chaque formulaire l'utilise.

2.2.3 Difficultés, solutions et état d'avancement

La principale difficulté a été de créer des composants suffisamment génériques pour couvrir les usages existants sans imposer un comportement trop rigide. Certains formulaires avaient des besoins proches, mais pas complètement identiques. Il fallait donc trouver un équilibre entre la centralisation du code et le respect des cas particuliers déjà présents dans l'application.

Après l'intégration, plusieurs problèmes sont apparus lors de la recette. Certaines dates ne pouvaient pas être saisies correctement au clavier, et des bordures d'erreur restaient affichées alors que le champ semblait corrigé. Ce dernier cas était le plus bloquant : l'état d'erreur persistait dans le formulaire et empêchait sa validation. Ces retours ont montré que la réussite d'une refactorisation ne se limite pas à la compilation du projet : il faut aussi vérifier les usages réels et les parcours complets.

Pour corriger ces problèmes, j'ai ajusté la gestion des saisies manuelles, la perte de focus et la synchronisation avec les formulaires réactifs. La gestion visuelle des erreurs a également été reprise afin que les bordures rouges disparaissent correctement lorsque la valeur redevient valide et que le formulaire puisse à nouveau être validé.

2.2.4 Retour de production et hotfix

Après la mise en production du refactoring, un comportement anormal a été signalé par un utilisateur sur la date de retour du formulaire de trajet. La sélection d'une date via le calendrier ne produisait pas toujours le même comportement que la saisie manuelle au clavier : la date pouvait rester bloquée sur la date du jour ou refuser une date antérieure, alors que la saisie directe dans l'input permettait de contourner le problème.

L'analyse du bug a mobilisé une grande partie de l'équipe. Même si je n'ai pas coordonné cette correction, j'ai pu participer à l'observation du comportement, aux échanges techniques et à la compréhension de la cause. Le problème venait d'une différence entre les événements déclenchés par les deux modes d'interaction. Lorsqu'un utilisateur quittait l'input, l'événement de perte de focus appelait bien `onTouched()`, ce qui indiquait au formulaire Angular que le champ avait été utilisé. En revanche, lors d'une sélection directe dans le calendrier,

l'événement `dateChange` mettait à jour la valeur sans toujours marquer le champ comme touché.

En résumé, le formulaire recevait bien la nouvelle date, mais l'état du champ n'était pas mis à jour de la même façon selon le mode de saisie. Angular ne considérait donc pas toujours le champ comme utilisé, ce qui pouvait perturber certaines validations et l'affichage des erreurs. La correction a consisté à appeler `onTouched()` lors du changement de date, afin que la sélection dans le calendrier ait le même comportement qu'une saisie clavier suivie d'une sortie du champ.

Comme le bug avait atteint la production, la correction a dû suivre la procédure de hotfix du projet. Il a fallu s'appuyer sur la documentation interne, relancer les pipelines nécessaires et échanger avec les personnes chargées des environnements et du déploiement. Cette situation m'a permis de mieux comprendre qu'une correction en production ne se limite pas au changement de code : elle implique aussi des validations, une coordination d'équipe et le respect d'un processus précis pour limiter les risques.

La mission est aujourd'hui terminée, avec des corrections intégrées après les retours de recette et une validation par un développeur de l'équipe. Le projet dispose désormais de deux composants partagés pour les champs de date et d'heure. Le résultat n'est pas seulement visuel : il réduit la duplication, rend le code plus homogène et facilitera les futures évolutions si l'équipe doit modifier le comportement ou le style de ces champs.

Au début de ce ticket, je pensais surtout déplacer du code dupliqué dans deux composants communs. En avançant, j'ai compris que la difficulté était plutôt de conserver exactement le même comportement dans des formulaires qui n'utilisaient pas tous les champs de la même manière. Ce travail m'a donc fait progresser sur les composants partagés, les formulaires réactifs, les bindings, la gestion des états et l'organisation du SCSS. Il m'a aussi appris à être prudent avec les refactorisations : même lorsqu'on ne change pas volontairement une règle métier, une modification technique peut provoquer des régressions si tous les cas d'usage ne sont pas vérifiés.

2.3 Mission 2 - Création et uniformisation des tableaux de gestion

2.3.1 Enjeux communs et démarche générale

Plusieurs tickets réalisés pendant la seconde partie du stage concernaient des tableaux de gestion de Flotto. Ils répondaient à des besoins métier différents, mais partageaient les mêmes enjeux : rendre les informations faciles à consulter, proposer une recherche et une pagination cohérentes, réutiliser les composants existants et limiter la duplication de logique ou de styles. J'ai donc choisi de les regrouper dans une même mission, car ils montrent une progression commune dans ma compréhension de l'architecture Angular et de la maintenabilité du projet.

Le tableau des motifs présent dans les paramètres constituait une référence utile pour l'organisation et le rendu visuel. Cependant, tous les tableaux ne pouvaient pas être rem-

placés directement par le composant générique `app-table`. Certains écrans possédaient des comportements spécifiques, comme la sélection multiple, des lignes extensibles ou la modification d'informations. L'objectif n'était donc pas de rendre tous les tableaux techniquement identiques, mais de mutualiser ce qui pouvait l'être tout en conservant leurs particularités fonctionnelles.

The image shows two screenshots of a web application interface. The top screenshot is titled 'Trip Reasons' and features a search bar with the placeholder 'Search a trip reason'. Below the search bar is a table with two columns: 'Label' and 'Precision'. The table contains five rows with labels: 'rdv client', 'raison 1', 'course', 'vacances', and 'depot vehicule'. Each row has a trash icon on the right. The table has alternating light blue and white row colors. Below the table is a pagination bar with the numbers 1, 2, 3, 4, and arrows for navigation. The bottom screenshot is titled 'Assigned sites' and also has a search bar with the placeholder 'Search an assigned site'. Below it is a table with one column: 'Assigned site'. The table contains six rows with labels: 'Agence Toulouse', 'Agence Rennes', 'Agence Paris', 'Agence Nantes', 'Agence Lyon', and 'Agence Lille'. Each row has a trash icon on the right. The table has alternating light blue and white row colors. Below the table is a pagination bar with the numbers 1, 2, and arrows for navigation.

Label	Precision
rdv client	
raison 1	
course	
vacances	
depot vehicule	

Assigned site
Agence Toulouse
Agence Rennes
Agence Paris
Agence Nantes
Agence Lyon
Agence Lille

Fig. 2.2 – Tableau des motifs utilisé comme référence visuelle, puis tableau des sites de rattachement reprenant la recherche, la pagination, les actions et l'alternance de couleur des lignes.

2.3.2 Création du tableau des sites de rattachement

Une première partie de cette mission, réalisée entre le début et la mi-juin 2026, concernait la création d'un tableau de gestion des sites de rattachement. Avant cette évolution, l'application utilisait des **lieux de dépôt** renseignés sous la forme d'un texte libre. Ces lieux indiquaient où les véhicules pouvaient être déposés, mais ils ne constituaient pas un référentiel structuré permettant de rattacher durablement chaque véhicule à un site précis.

De plus, un même lieu pouvait être saisi de différentes manières, ce qui rendait les données moins homogènes et plus difficiles à exploiter.

L'évolution globale devait donc introduire une nouvelle notion : le **site de rattachement**. Contrairement au lieu de dépôt saisi librement, ce site provient d'un référentiel configurable par les gestionnaires de flotte. Il peut ensuite être associé aux véhicules afin d'améliorer la cohérence des données et de faciliter leur filtrage lors des réservations ou de la création de trajets.

Cette évolution était répartie entre plusieurs tickets. Ma partie portait sur la consultation et la suppression des sites depuis la page Paramètres ; leur création appartenait à un autre ticket. Le tableau devait permettre de rechercher, trier et paginer les sites, puis de demander leur suppression depuis chaque ligne. Une modale évitait les suppressions accidentelles et une notification informait l'utilisateur du résultat de l'opération. J'ai également ajouté les traductions françaises et anglaises ainsi qu'une infobulle présentant l'utilité des sites.

L'organisation retenue séparait plusieurs responsabilités. Le composant parent pilotait la recherche, le tri, la pagination et la suppression. Le composant du tableau se concentrait sur l'affichage des colonnes et des actions disponibles. Un store conservait l'état de la page, tandis qu'un service assurait les échanges HTTP avec l'API. Cette séparation rendait le fonctionnement plus lisible et évitait de mélanger la présentation, l'état de l'interface et les appels au backend.

Le ticket demandait également d'utiliser et de documenter les **signals** Angular. Cette approche n'était pas encore utilisée dans Flotto, dont la gestion d'état reposait principalement sur des observables RxJS. La mission devait donc servir de premier exemple documenté pour de futures utilisations dans l'application. Les **signals** conservaient les sites affichés, la recherche, le tri et la page courante. Des valeurs **computed** construisaient automatiquement les informations nécessaires à la pagination lorsque cet état évoluait, tandis que RxJS restait principalement utilisé autour des appels HTTP.

La difficulté principale venait des dépendances avec d'autres tickets encore en cours de validation. Le service frontend et les routes backend nécessaires étaient développés en parallèle par d'autres membres de l'équipe. L'utilisation des **signals** représentait également une difficulté, car cette approche était nouvelle pour moi et aucun exemple n'existait encore dans le projet. Enfin, les premiers tests avec le backend ont révélé des écarts entre les noms et les routes attendus par le frontend et ceux réellement exposés, ainsi qu'un tri qui n'était pas encore appliqué côté serveur.

Pour avancer sans perturber le travail des autres développeurs, j'ai créé des branches locales temporaires réunissant les tickets dépendants. J'ai d'abord utilisé un service simulé, puis créé des données directement via l'API pour tester le fonctionnement réel. L'onglet Network du navigateur m'a permis de contrôler les requêtes et de signaler les écarts aux développeurs concernés. L'implémentation frontend est terminée et testée localement ; son intégration définitive reste dépendante de la fusion des tickets associés.

2.3.3 Évolution de la page Membres et uniformisation visuelle

Une autre partie de la mission concernait la page de gestion des membres. Celle-ci regroupait initialement les membres actifs et les invitations en attente sans séparation suffisamment claire. J'ai réorganisé la page autour de trois onglets : **Membres de la flotte**, **Invitations envoyées** et **Demandes d'intégration**. Chaque onglet possède une icône, un compteur, une barre de recherche commune et une pagination limitée à cinq lignes.

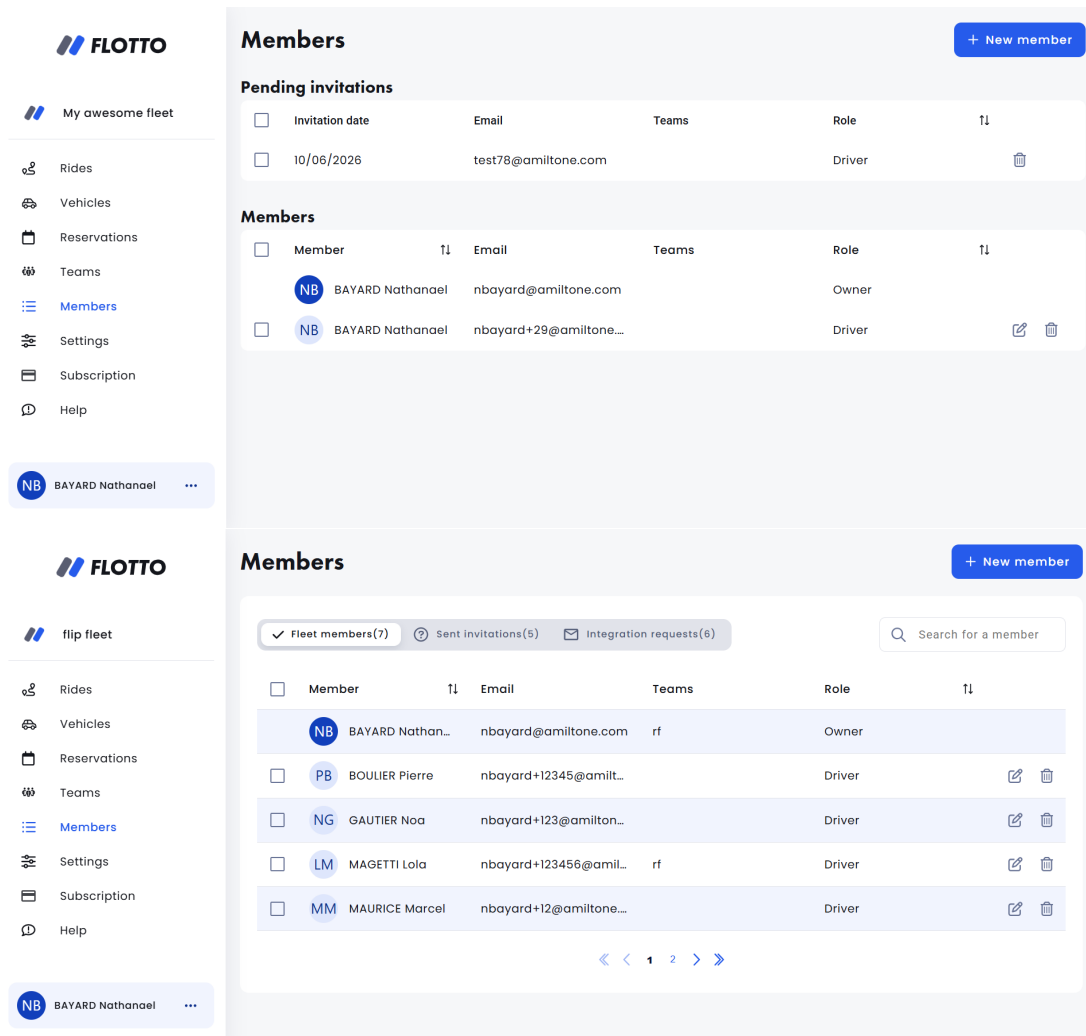


Fig. 2.3 – Évolution de la page Membres : organisation initiale regroupant les tableaux sur une même page, puis nouvelle navigation par onglets avec recherche et pagination communes.

La route backend permettant de récupérer les demandes d'intégration n'étant pas encore disponible, le troisième onglet a été préparé avec des données simulées. J'ai créé le modèle associé, le service de filtrage et de pagination ainsi que le tableau affichant le membre demandeur, son adresse électronique et la date de sa demande. La logique a été répartie entre plusieurs services afin d'alléger le composant principal. Des états vides ont aussi été ajoutés : lorsqu'une recherche ne retourne aucun résultat, le tableau et sa pagination laissent place à un message centré, conformément aux pratiques déjà utilisées dans d'autres pages.

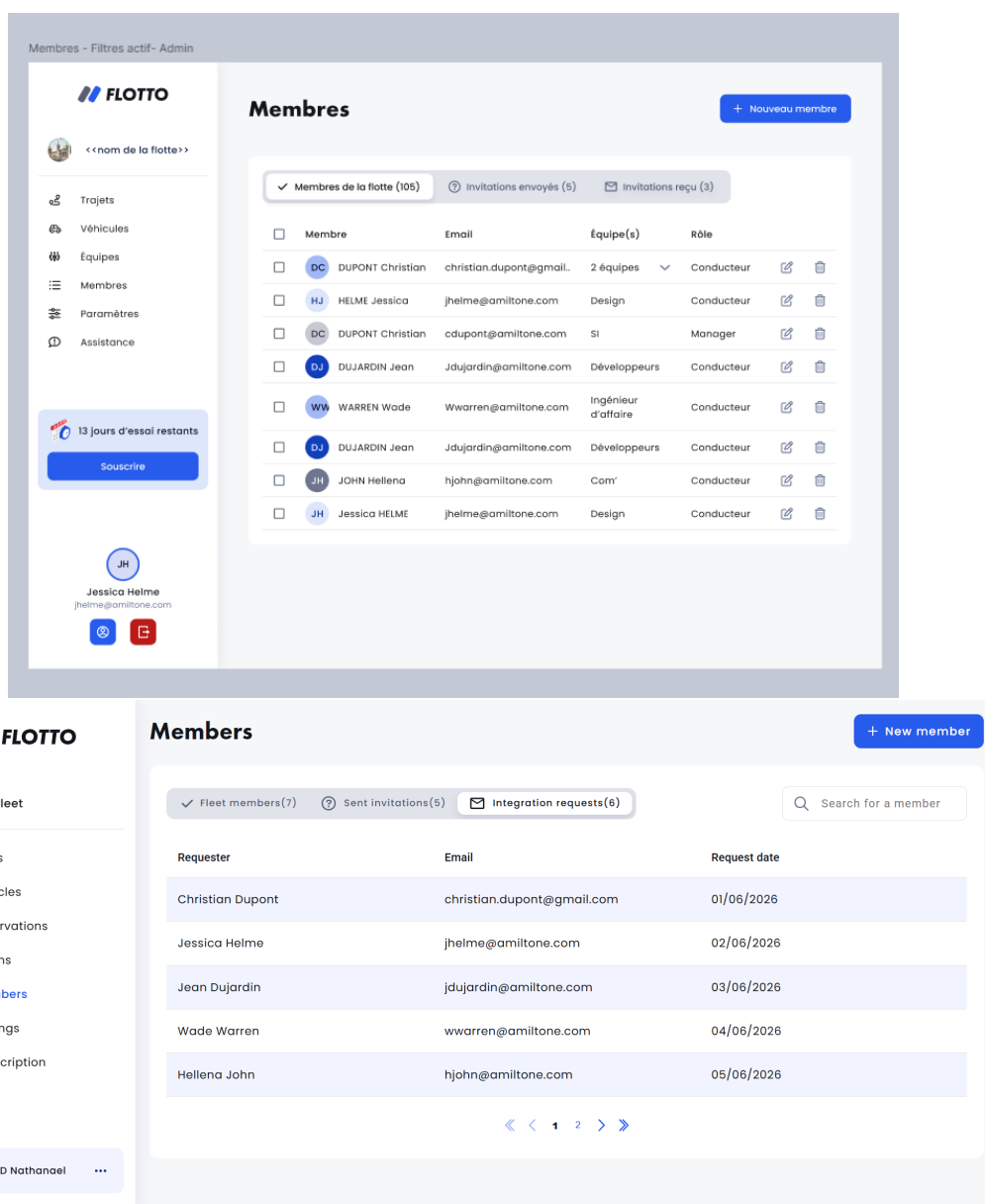


Fig. 2.4 – Maquette de la nouvelle organisation de la page Membres et implémentation de l'onglet Demandes d'intégration avec des données simulées, dans l'attente du futur endpoint backend.

Une part importante du travail concernait l'évolution du design des tableaux. Au début du développement, le tableau des membres avait été adapté au plus près de la maquette fournie, avec des règles SCSS propres à cette page. Après échange avec l'équipe, la décision a été prise d'uniformiser l'apparence des tableaux de l'application en prenant celui des motifs comme référence, notamment pour l'alternance de couleur des lignes et leur changement de couleur au survol.

Il n'était toutefois pas possible de copier simplement le fichier SCSS des motifs ou de remplacer les tableaux de la page Membres par `app-table`. Ces tableaux gèrent des cases à cocher, la sélection et la suppression multiple, des colonnes d'actions, des lignes extensibles présentant plusieurs équipes et la modification du rôle d'un membre. J'ai donc extrait les

règles visuelles réellement communes dans des mixins SCSS, puis refactoré les styles des motifs et des membres afin qu'ils utilisent ces mixins tout en conservant leurs comportements propres.

Ce refactoring m'a demandé de mieux comprendre la cascade CSS et plusieurs anciennes règles de largeur ou d'alignement. Le nouveau tableau des demandes d'intégration ne contenait par exemple que trois colonnes, mais héritait initialement de proportions prévues pour les tableaux à six colonnes. Il a fallu identifier l'origine de ces styles et les adapter sans provoquer de régression sur les tableaux existants.

J'ai également corrigé un problème lié à la suppression multiple. Les éléments étaient bien supprimés, mais l'interface pouvait conserver le bouton de suppression et afficher une notification d'erreur. L'utilisation de `forkJoin` a permis d'attendre la fin de toutes les requêtes avant de recharger les données et d'afficher une seule notification cohérente.

2.3.4 Résultats et apprentissages

Ces réalisations ont abouti à des tableaux plus cohérents, aussi bien dans leur fonctionnement que dans leur présentation. Le tableau des sites de rattachement est fonctionnel dans l'environnement local d'intégration et introduit une première utilisation documentée des `signals` dans Flotto. La page Membres dispose désormais d'une navigation plus claire, d'une recherche et d'une pagination communes, d'états vides cohérents et d'un troisième onglet prêt à être connecté au futur endpoint backend.

L'uniformisation des styles réduit la duplication du SCSS et facilite les futures évolutions visuelles, sans supprimer les comportements spécifiques des tableaux complexes. Les modifications ont été vérifiées avec des tests unitaires Jest, le contrôle TypeScript, Prettier, la compilation Angular et les contrôles automatiques Husky. Sur cette mission, j'ai surtout retenu qu'un tableau peut sembler simple visuellement, alors qu'il concentre en réalité beaucoup de logique : état de la recherche, pagination, appels backend, styles hérités et comportements propres à chaque page.

2.4 Missions ponctuelles ou récurrentes

En dehors de ces deux missions, j'ai également traité de petits tickets liés à la prise en main du projet, à la refactorisation et à l'amélioration de la qualité du code, notamment dans le contexte d'une montée de version d'Angular permettant d'utiliser de nouvelles écritures. Ces tickets ont contribué à ma progression, mais ils ne sont pas suffisamment significatifs pour être détaillés davantage.

Le traitement de ces tickets suit déjà une démarche professionnelle : lecture du besoin, recherche dans le code existant, modification sur une branche dédiée, vérification locale, puis préparation d'une merge request lorsque le ticket est prêt à être relu. Même lorsqu'un ticket est de taille limitée, il oblige à respecter les conventions du projet et à comprendre l'impact.

Chapitre 3

Organisation du travail, résultats et apports

3.1 Gestion du projet et calendrier des missions

L'organisation du travail au Nexus suit une logique agile adaptée aux produits internes. Les sprints structurent l'avancement des équipes : les tickets sont préparés, priorisés, développés, testés, puis présentés lors de démonstrations. La fin de sprint permet aussi de stabiliser le livrable, de faire remonter les retours, de préparer les priorités suivantes et d'identifier des actions d'amélioration.

Plusieurs rituels rythment le travail. Le daily permet de partager l'avancement et les blocages. Le weekly Nexus donne une vision plus globale des différents produits et des annonces communes. Les démonstrations et reviews permettent de présenter les évolutions livrables. Les rétrospectives servent à identifier ce qui a bien fonctionné, les difficultés et les améliorations possibles. Enfin, le sprint planning et les réunions de chiffrage permettent de sélectionner et d'estimer les tickets suivants.

Le calendrier ci-dessous présente les grandes phases de mon stage. Il ne reprend pas tout le contenu du board Jira, mais il permet de situer ma progression, les missions réalisées et leur place dans l'organisation du projet.

Période	Activités principales	Résultats
Semaine 1	Découverte d'Amiltone et du Nexus, lecture de la documentation interne, installation du poste de travail.	Environnement de travail opérationnel et première compréhension des outils.
Semaine 2	Intégration à l'équipe Flotto, découverte du projet et première prise en main d'Angular.	Compréhension initiale de l'application, de son architecture et du workflow de l'équipe.
Semaines 3 à 4	Montée en compétences sur Angular et traitement de premiers tickets de refactorisation ou de qualité de code.	Premières contributions au projet et meilleure lecture de l'existant.

Période	Activités principales	Résultats
semaine 5 et plus	Réalisation du ticket : refactorisation des champs de date et d'heure en composants réutilisables.	Composants partagés créés, formulaires mis à jour, corrections après recette et validation par un développeur de l'équipe.
Après mise en production du ticket	Participation à l'analyse d'un bug utilisateur sur la date de retour du formulaire de trajet et suivi de la procédure de hotfix avec l'équipe.	Correction appliquée, pipelines relancés et meilleure compréhension des contraintes d'une correction en production.
Début à mi-juin	Création du tableau de consultation et de suppression des sites de rattachement dans la page Paramètres.	Frontend terminé et testé localement ; intégration définitive en attente des tickets associés.
Mi-juin	Réorganisation de la page Membres et uniformisation visuelle des tableaux de gestion.	Navigation par onglets, styles SCSS mutualisés et correction de la suppression multiple.
Suite du stage	Poursuite des tickets attribués, participation aux rituels agiles, tests ponctuels.	Contributions progressives aux sprints et consolidation des compétences techniques et professionnelles.

3.2 Résultats obtenus

Le résultat le plus concret de cette première mission est la mise en place de deux composants partagés : `app-date-input` et `app-time-input`. Ils remplacent plusieurs utilisations directes de `mat-datepicker` et `mat-timepicker` dans les formulaires concernés.

Cette contribution ne modifie pas le périmètre fonctionnel de Flotto, mais elle améliore l'homogénéité des champs de date et d'heure et réduit la duplication de code.

Le retour de production sur la date de retour a également permis de compléter la correction du composant `app-date-input`. Il a mis en évidence l'importance de tester un même champ selon plusieurs modes d'interaction, car une saisie clavier et une sélection via calendrier peuvent déclencher des états Angular différents.

La deuxième mission a produit plusieurs résultats complémentaires. Le tableau des sites

de rattachement permet de consulter, rechercher, trier, paginer et supprimer les sites ; son intégration finale reste dépendante d'autres tickets, mais il a pu être validé avec le backend dans un environnement local d'intégration.

La page Membres dispose désormais d'une navigation par onglets plus claire et d'un troisième tableau prêt à être connecté au futur endpoint backend. Les tableaux concernés partagent également des règles SCSS communes, tout en conservant leurs comportements spécifiques. La correction de la suppression multiple rend enfin le retour affiché à l'utilisateur plus cohérent.

En parallèle de ces deux missions, j'ai aussi traité d'autres tickets plus ponctuels, principalement liés à la prise en main du projet, à la refactorisation et à la qualité du code.

3.3 Apports pour l'entreprise

Mon apport reste modeste à l'échelle du projet, mais il s'inscrit dans le travail collectif de l'équipe Flotto. Il passe par le traitement de tickets, des vérifications locales, des corrections après retours et la participation aux rituels de sprint.

La première mission améliore la maintenabilité du projet en centralisant le comportement des champs de date et d'heure. La seconde apporte de nouvelles possibilités de gestion tout en améliorant la cohérence des tableaux existants. La mutualisation des styles SCSS facilite notamment les futures évolutions visuelles et limite les différences entre les pages. À mon niveau de stage, ces contributions participent donc à l'amélioration progressive de la qualité du produit.

3.4 Limites et pistes de suite

La principale limite rencontrée pendant le stage a été mon manque d'expérience sur Angular et sur un projet professionnel de cette taille. L'application comportait déjà beaucoup d'existant et demandait un temps d'adaptation important. Il a donc fallu accepter une progression graduelle : comprendre avant de modifier, poser des questions lorsque c'était nécessaire et éviter de se lancer trop rapidement sans avoir identifié les impacts possibles.

Une autre limite vient du fait que la documentation interne est riche mais parfois hétérogène. Certaines informations concernent l'organisation générale du Nexus, d'autres un produit précis, une ancienne version ou un processus interne. J'ai donc dû apprendre à croiser les sources, vérifier ce qui était encore actuel et ne retenir que ce qui était utile pour le travail en cours.

Les deux missions détaillées dans ce rapport illustrent des situations complémentaires. La première porte sur la création de composants partagés et la maintenance d'un existant. La seconde associe réalisation fonctionnelle et refactoring transversal : certaines parties dépendent encore de futurs endpoints ou de tickets développés en parallèle, tandis que les styles mutualisés devront être réutilisés progressivement par d'autres tableaux.

Chapitre 4

Bilan de compétences et projet professionnel

4.1 Compétences techniques mobilisées

Sur le ticket de refactorisation des champs de date et d'heure, j'ai principalement mobilisé la compétence **Réaliser**. J'ai dû lire un existant Angular, créer deux composants réutilisables, les intégrer dans plusieurs formulaires et vérifier leur comportement.

Ce travail m'a aussi fait mesurer ce que représente la maintenance d'une application existante. L'objectif n'était pas d'ajouter une fonctionnalité visible, mais d'améliorer la structure du code sans modifier les règles métier déjà présentes. Cela m'a obligé à être attentif aux impacts possibles d'un changement sur plusieurs formulaires.

Avec les tableaux de gestion, le travail a été différent. Le tableau des sites de rattachement m'a amené à utiliser les **signals** Angular, à séparer l'état de la page des appels HTTP et à vérifier les échanges entre le frontend et le backend. Le travail sur la page Membres m'a fait approfondir le SCSS, la cascade CSS et la mutualisation de styles à l'aide de mixins, tout en conservant les comportements propres à des tableaux complexes.

Les autres tickets et vérifications réalisés pendant le stage m'ont aussi permis de renforcer ma lecture du code existant. Même lorsqu'une tâche était limitée, elle demandait de comprendre le module concerné, d'identifier les fichiers à modifier, de respecter les conventions Angular du projet et de vérifier que le comportement attendu restait correct.

Enfin, la validation a occupé une place importante. Les essais en local, les retours de recette et les corrections m'ont montré qu'un développement ne s'arrête pas lorsque le code compile : il doit aussi être testé dans les usages réels de l'application.

D'autres compétences ont été mobilisées de manière plus transversale. L'utilisation de Jira, des sprints et des daily m'a permis de mieux comprendre la conduite d'un projet informatique. Les échanges avec l'équipe, les retours de recette et les demandes d'aide ont aussi renforcé la compétence **Collaborer**. Enfin, l'installation et l'utilisation de l'environnement de développement m'ont fait travailler certains aspects liés à l'administration d'un poste de développement.

4.2 Compétences transversales développées

Au-delà des aspects techniques, le stage m'a aussi demandé de progresser dans ma manière de travailler en équipe, de m'organiser et de communiquer mon avancement.

Ces compétences existaient déjà dans les projets réalisés à l'IUT, mais le contexte professionnel les rend plus concrètes : les tickets s'inscrivent dans un produit existant, les échanges concernent une équipe réelle et les validations ont un impact direct sur le sprint en cours.

L'autonomie est nécessaire pour lire la documentation, chercher dans le code, comprendre un ticket et avancer avant de demander de l'aide. Cependant, cette autonomie ne signifie pas travailler seul : elle suppose aussi de savoir formuler une question claire, indiquer ce qui a déjà été essayé et solliciter la bonne personne au bon moment.

L'organisation personnelle est également importante. Le travail par tickets oblige à suivre l'état des tâches, à respecter les priorités du sprint et à garder une trace de ce qui a été fait. Les rituels d'équipe, comme le daily, les démonstrations ou les réunions de chiffrage, m'aident à comprendre comment communiquer un avancement de manière courte et utile.

Enfin, l'intégration dans une équipe professionnelle demande de s'adapter à des habitudes existantes. Il faut accepter une phase d'observation, comprendre le vocabulaire du projet, respecter les conventions et tenir compte des contraintes du produit et de l'équipe. Ces éléments sont moins visibles qu'une fonctionnalité livrée, mais ils conditionnent la qualité du travail réalisé.

4.3 Difficultés formatrices et progression personnelle

La principale difficulté est la découverte simultanée de plusieurs éléments nouveaux : Angular, le projet Flotto, les outils professionnels, l'organisation du Nexus et les attentes de qualité. Dans un cadre scolaire, les projets sont souvent plus courts et mieux délimités. Ici, il faut comprendre un existant plus large, parfois sans connaître toutes les décisions prises auparavant.

Avec le recul, cette difficulté a surtout changé ma manière d'aborder un ticket. Avant de modifier du code, je dois prendre le temps de lire, chercher les usages existants, vérifier les impacts possibles et comprendre le besoin. Je progresse aussi dans ma manière de demander de l'aide : une question précise, appuyée sur ce que j'ai déjà compris, permet d'obtenir une réponse plus efficace.

La configuration initiale de l'environnement constitue un exemple concret de cette progression. Le projet repose notamment sur WSL et Docker, que je connaissais peu avant le stage, mais qui sont indispensables pour lancer Flotto et travailler dans les mêmes conditions que l'équipe. Leur prise en main m'a demandé de suivre la documentation interne, de comprendre le rôle des différents services et de solliciter de l'aide lorsque mes recherches ne suffisaient pas. Une fois l'environnement opérationnel, j'étais davantage capable d'identifier l'origine d'un problème et d'expliquer précisément les vérifications déjà effectuées.

Certains tickets représentaient une autre difficulté, car leur description supposait parfois une connaissance du produit ou de décisions prises auparavant. Plutôt que de commencer une modification sur une interprétation incertaine, j'ai appris à parcourir les écrans et le code concernés, à reformuler le besoin puis à demander confirmation à l'équipe. Cette démarche m'a permis de poser des questions plus utiles et de gagner progressivement en autonomie sans risquer de développer une solution ne correspondant pas au besoin attendu.

4.4 Lien avec la formation et le projet professionnel

Le stage est en lien direct avec la formation du BUT , en particulier avec le parcours RA. Les notions de développement web, de qualité, de gestion de projet, de travail en équipe et de validation prennent ici une forme concrète. Les outils utilisés en entreprise, comme Jira, GitLab, Docker ou les environnements de validation, donnent une dimension professionnelle à des pratiques déjà abordées pendant la formation.

Cette expérience me permet aussi de mieux comprendre ce que signifie rejoindre un projet logiciel existant. Le développement n'est pas seulement une activité technique individuelle : il dépend d'une organisation, de rituels, de choix collectifs, d'une documentation et de contraintes produit. Pour la suite de mon parcours, ce stage doit m'aider à mieux identifier les compétences à consolider, notamment sur Angular, les tests, la qualité du code et la communication en équipe.

Avant ce stage, j'hésitais encore entre poursuivre mes études jusqu'à un niveau bac +5 et chercher directement un emploi après le BUT. Cette expérience me conforte davantage dans l'idée d'entrer dans la vie professionnelle après la troisième année. Travailler sur un projet concret, au sein d'une équipe et avec de réelles contraintes techniques m'a montré que l'expérience acquise en entreprise constitue également une manière importante de continuer à progresser. Cette orientation reste à confirmer au cours de ma troisième année, mais je souhaite désormais privilégier la recherche d'un premier emploi afin de renforcer mes compétences par la pratique.

Conclusion

En arrivant chez Amiltone, je ne savais pas encore précisément à quoi ressemblait le quotidien d'une équipe de développement sur un produit déjà en production. Le stage m'a permis de le découvrir de manière concrète, en participant au projet Flotto et en traitant progressivement des tickets. Même si la convention mentionnait initialement le développement mobile avec Flutter, je n'ai pas eu le temps de commencé que j'ai directement été recentré sur le développement web avec Angular, ce qui correspondait mieux au contexte et aux besoins de l'équipe.

Les deux missions principales résument les grands types de travaux réalisés pendant le stage. Sur les champs de date et d'heure, j'ai surtout travaillé sur la maintenance et la fiabilisation d'un existant déjà utilisé. Sur les tableaux de gestion, j'ai davantage participé à des évolutions visibles, avec de nouvelles interfaces, l'introduction des `signals` Angular dans Flotto et la mutualisation de styles SCSS. Les difficultés rencontrées, notamment le retour de production sur les champs de date, les dépendances entre plusieurs tickets et l'absence de certains endpoints backend, m'ont appris à tester plus largement mes modifications et à communiquer avec l'équipe lorsqu'un problème dépassait mon propre périmètre.

Dans l'ensemble, les attentes formulées au début du stage ont été satisfaites. Je ne considère pas avoir tout maîtrisé, notamment sur Angular ou sur l'architecture complète de Flotto, mais j'ai progressé dans ma capacité à comprendre puis modifier du code existant. J'ai également gagné en autonomie, tout en comprenant qu'elle repose autant sur la recherche personnelle que sur la capacité à demander de l'aide au bon moment. Par rapport aux projets réalisés à l'IUT, cette expérience m'a surtout apporté une meilleure compréhension de la maintenance, de la non-régression et du travail collectif autour d'un produit destiné à de réels utilisateurs.

Cette expérience confirme mon intérêt pour le développement d'applications et la cohérence de mon parcours RA. Pour la suite du BUT, je souhaite approfondir les compétences encore fragiles, notamment sur Angular, les tests automatisés et l'architecture d'applications existantes. Le stage constitue ainsi une première expérience professionnelle importante, qui me permettra d'aborder les prochains projets avec davantage de méthode, de recul et de confiance.

Sources

Les informations utilisées pour ce rapport proviennent principalement des sources suivantes :

- livret de stage et consignes pédagogiques de l'IUT ;
- documentation interne Amiltone/Nexus, uniquement sous forme de synthèse ;
- synthèse de la documentation Flotto ;
- référentiel de compétences du BUT Informatique, parcours Réalisation d'applications : conception, développement, validation ;